C200 Programming Assignment № 4

Dr. M.M. Dalkilic
Computer Science
School of Informatics, Computing, and Engineering

Indiana University, Bloomington, IN, USA

October 12, 2024

Introduction

Due Date: 11:00 PM, Friday, October 18, 2024 Submit your work to the Autograder `https://c200.luddy.indiana.edu/` and remember to add, commit and **push often** to GitHub **before the deadline. We do not accept late submissions.**

In this homework, you'll start mastering your skill in writing functions. Make sure to start working early on this assignment. If we do not explictly mentioned a function/library to use, you are not allowed to use it. For example, if we have not mentioned to use tkinter library then you should not import tkinter in your python file.

Note: If you're found cheating...well, you know the consequences by now which includes using tools to solve *any* portion of any problem.

Instructions

1. Make sure that you are **following the instructions** in the PDF, especially the format of output returned by the functions. For example, if a function is expected to return a numerical value, then make sure that a numerical value is returned (not a list or a dictionary). Similarly, if a list is expected to be returned then return a list (not a tuple, set or dictionary).
2. Test **debug** the code well (syntax, logical and implementation errors), before submitting to the Autograder. These errors can be easily fixed by running the code in VSC and watching for unexpected behavior such as, program failing with syntax error or not returning correct output.
3. Make sure that the **code does not have infinite loop (that never exits)** before submitting to the Autograder. You can easily check for this by running in VSC and watching for program output, if it terminates timely or not.
4. Given that you already tried points 1-3, if you see that Autograder does not do anything (after you press 'submit') and waited for a while (30 seconds to 50 seconds), try refreshing the page or using a different browser.

5. Once you are done testing your code, comment out the tests i.e. the code under the __name__ == "__main__" section.

Problem 1: Day of the Week

The modulus operator is denoted by the symbol %. For $x\%y$ it returns the remainder after x is divided into y a whole number of times. While you've seen this before, the format is likely different. To refresh your memory you can visit `https://realpython.com/python-modulo-operator/` or do your own search. For this problem, you will also need to use the floor function in python - think of floor as a way to round down a floating point number to the nearest integer. For example, floor(18.90) is 18 and floor(14.25) is 14.

An approximate (works generally okay for a number of dates, but **not** all) formula for computing the day of the week given dlst = [d,m,y] where d is day, m is month, and y is year is:

$$dlst = [d, m, y] \tag{1}$$

$$a(dlst) = y - \frac{(14 - m)}{12} \tag{2}$$

$$x = a(dlst) + \frac{a(dlst)}{4} - \frac{a(dlst)}{100} + \frac{a(dlst)}{400} \tag{3}$$

$$b(dlst) = floor(x) \tag{4}$$

$$c(dlst) = m + 12(\frac{14 - m}{12}) - 2 \tag{5}$$

$$day((d, m, y)) = (d + b(dlst) + (31 \cdot \frac{c(dlst)}{12}))\%7 \tag{6}$$

The function day returns a number $1, 2, \ldots, 7$ where $1 = Monday, 2 = Tuesday, \ldots$. If we use this number as the key with the week dictionary, we are able to return the correct day of the week. Here is the dictionary used:

```
1 week = {1:"Mon", 2:"Tue", 3:"Wed", 4:"Thu", 5:"Fri", 6:"Sat", 7:"Sun"}
```

For example,

$$day([14, 2, 2000]) = \text{Mon} \tag{7}$$

$$print(day([14, 2, 1963])) = \text{Thu} \tag{8}$$

$$print(day([14, 2, 1972])) = \text{Mon} \tag{9}$$

2/14/2000 falls on a Monday; 2/14/1963 falls on a Thursday; 2/14/1972 falls on a Monday.

Deliverables for Problem 1

- Complete the functions described above.
- For calculating b(dlst), you can use the math.floor() function from the math library.
- Remember, the day function utilizes the week dictionary.

Problem 2: Tree Sap

You've been contracted by a law firm to establish ownerhip of property. You've been given a model for tree growth (height in feet) as a function of time (years):

$$h(t) = \frac{120}{1 + 200e^{\alpha t}} \tag{10}$$

$$\alpha = -.2 \tag{11}$$

A dispute involves an owner who claims he planted some trees when his family first acquired the land 20 years ago. You're given permission to measure the height of the two tallest trees measuring $50\,\text{ft}$ and $100\,\text{ft}$. You know the model isn't precise, and so you decide if the time is within half a decade it's probably accurate enough. For example, A tree that is $50\,\text{ft}$ is about 24.81 years old, then if someone claims to have planted it 20 years ago:

$$20 - 5 \leq 24.8 \leq 20 + 5 \tag{12}$$

which is true. On the other hand, if the tree is $100\,\text{ft}$, then

$$20 - 5 \leq 34.54 \leq 20 + 5 \tag{13}$$

which is false. Using the formula for $h(t)$ implement the inverse function, *i.e.*, given a height, what is the time that has elapsed since it was planted. The following code:

```
1 print(tree_age(50,20))
2 print(tree_age(100,20))
```

has the output:

```
1 (24.81, True)
2 (34.54, False)
```

where the tuple's first component is the life of the tree and the Boolean whether this is within the interval.

Deliverables for Problem 2

- Complete the function.
- Round to two decimal places.

Problem 3: Using Loop to Calculate Sinking Fund Schedule

A **sinking fund** is an financial instrument to discharge a future debt. For example, a manufacturing company expects to replace obsolescent machinary in some number of years. In this problem, you are building your finance company to compute payment schedules for a sinking fund. We use the general formula for annuity:

$$\begin{aligned} S &= R\frac{(1+i)^n - 1}{i} \\ i &= \frac{r}{m} \\ n &= my \end{aligned}$$

where S is the future debt, r the (percentage) interest rate, m the number of payments per year, y the number of years, n is the total number of payments (number of payments in a year multiplied by total number of years) and R is the monthly deposit made. Here is the particular problem. Acme Sundials believes replacing their sun dial mold-press will cost $30,000 in two years. They can currently get 10% interest compounded quarterly (four times a year). You build a Python program that, given the initial data will:

- Determine the monthly deposit (R).
- Determine the total schedule of payments that includes the interest accrued and total amount. This should be stored as a list of lists (see example output below).

Here is the run:

```
1 def deposit(S,r,i,n):
2     pass
3
4 def sinking_fund(final_amt,r,m,y):
5     pass
6
7 for i in sinking_fund(30000,.1,4,2):
8     print(i)
```

with output:

```
1 [[0, 3434.02, 0, 3434.02],
2 [1, 3434.02, 85.85, 6953.89],
3 [2, 3434.02, 173.85, 10561.76],
4 [3, 3434.02, 264.04, 14259.82],
5 [4, 3434.02, 356.5, 18050.34],
6 [5, 3434.02, 451.26, 21935.62],
7 [6, 3434.02, 548.39, 25918.03],
8 [7, 3434.02, 647.95, 30000.0]]
```

- The function deposit returns the monthly deposit needed for the fund (R).
- Clearly you must solve for R (monthly deposit) first and use this value in sinking_fund() to solve for S.
- Once you have the deposit value, you should find n and i.
- After you have all the values, i..e, i, n, R, you can start calculating your payment schedule.
- Make sure that the payment schedule is stored as a list of lists where each sublist represent one payment as follows: [period, deposit, interest, total fund].

Observe there are eight payments. On the last payment, the fund is $30,000. The first value in the list is the period, the second is the deposit amount, the third is the interest accrued on the **previous** fund, the last value is the curent total fund. Let's construct period 1 from 0. The total fund in period 0 is $3434.02, or the initial deposit value for this particular example.

$$\begin{aligned} i \times fund &= \frac{r}{m} \times 3434.02 \\ &= \frac{.10}{4} \times 3434.02 = 0.025(3434.02) = 85.85 \end{aligned}$$

The total t_1 for that period is the previous total, plus interest, plus deposit:

$$t_1 = 3434.02 + 85.85 + 3434.02 = 6953.89$$

Deliverables for Problem 3

- Complete the functions deposit and sinking_fund.
- Round the returned value to 2 decimal places in both functions.
- You are free to use a single bounded (hint), unbounded, nested loops–whatever you find most comfortable.

Problem 4: Space, the Final Frontier

The weight of an astronaut whose altitude is d kilometers above the ground weighs approximately:

$$A_w(E_w, d) \quad = \quad E_w \frac{6400}{6400 + d} \tag{14}$$

where E_w is the weight on earth. Implement a function that can give the minimal altitude an astronaut must attain given both the weight on earth and distance from the earth. For example, a 165 pound (earth weight) will weigh 5 pounds if $d \geq 30,366$ kilometers from the earth's surface. The following code:

```
1 print(orbit(5,165))
```

produces

```
1 30366
```

Deliverables for Problem 4

- Complete the function.
- Use the math ceiling function for the solution.

Problem 5: Come Fly with Me (Sinatra)

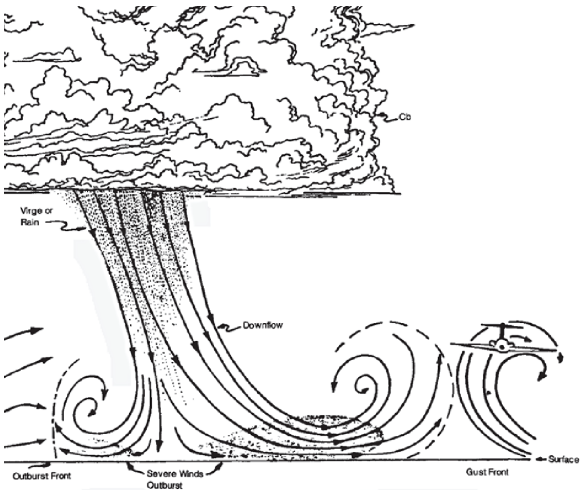

Figure 1: Image taken from FAA

You've been hired by the FAA to calculate wind shear contants. Wind shear is a meterological phenomenon where the velocity of wind v_0 at height h_0 and the velocity of wind v_1 at height h_1 differ significantly. Wind shear affects airplanes as they take-off and land. The formula for verticle wind sheer is:

$$\frac{v_0}{v_1} = (\frac{h_0}{h_1})^P \tag{15}$$

Implement a function P(v0,h0,v1,h1) that takes wind velocities and heights and returns the constant value of P. The following function:

```
v0,h0 = 25,200
v1,h1 = 6,35
print(P_ws(v0,h0,v1,h1))
```

produces

```
0.819
```

Deliverables for Problem 5

- Complete the function.
- Round to three places.

Problem 6: Approximation

In this problem, we'll implement approximations to functions that are described by infinite series. Given a function $f(x)$ and a value $x*$, a Taylor series is the value $f(x*)$ requiring a sum of infinite ratios determined by the derivative of $f(x)$. In this problem we are not interested in how the Taylor series is derived; rather, we want to leverage it to approximate a value.

The first function we study is $f(x) = e^x$:

$$e^x = \sum_{n=0}^{\infty} \frac{x^n}{n!} \tag{16}$$

$$\approx 1 + x + \frac{x^2}{2!} + \cdots + \frac{x^\tau}{\tau!} \tag{17}$$

where $\tau \in \mathbb{N}$ and functions as number of terms. For example, to approximate, $e^2 = 7.38905609893065$. Let's look at $\tau \in \{1, 5, 10\}$:

$$\sum_{n=0}^{1} \frac{2^n}{n!} = 1 + 2 = 3 \tag{18}$$

$$\sum_{n=0}^{5} \frac{2^n}{n!} = 1 + 2 + \frac{2^2}{2!} + \frac{2^3}{3!} + \frac{2^4}{4!} + \frac{2^5}{5!} \tag{19}$$

$$= 1 + 2 + \frac{4}{2} + \frac{8}{6} + \frac{16}{24} + \frac{32}{120} \tag{20}$$

$$\approx 1 + 2 + 2 + 1.333 + 0.666 + 0.266 = 7.26659 \tag{21}$$

$$\sum_{n=0}^{10} \frac{2^n}{n!} = 7.388994708994708 \tag{22}$$

The following code:

```
print(math.exp(2), e_(2,1), e_(2,5), e_(2,10))
print(math.exp(3), e_(3,50))
```

produces

```
7.38905609893065 3.0 7.266666666666667 7.388994708994708
20.085536923187668 20.08553692318766
```

We'll implement a function `e_(x,tau)` that sums the term $\frac{x^n}{n!}$, `tau + 1` times (because we're including zero). Additionally, we'll implement a helper function `term(x,n)` that returns the actual term in the series. So we'll have:

```
def e_(x,tau):
    def term(x,n):
    pass
    pass
```

for this function.

Approximating the cos function Trigonometric functions are determined from Taylor series. With each term, the curve is better approximated with a polynomial. We generally use polyonomials to approximate complex curves, because polynomials are simple to describe and possess nice properties (that we'll discuss later). For this problem, we'll look at cosine:

$$\cos(x) \quad = \quad \sum_{n=0}^{\infty} \frac{(-1)^n}{(2n)!} x^{2n} \tag{23}$$

For example, if $\tau = 2$, we can see in Fig. 2 that in the interval [-1.3,1.3] the curves are the same, but outside the interval, they diverge. The Table 1 shows some values for x allowing us

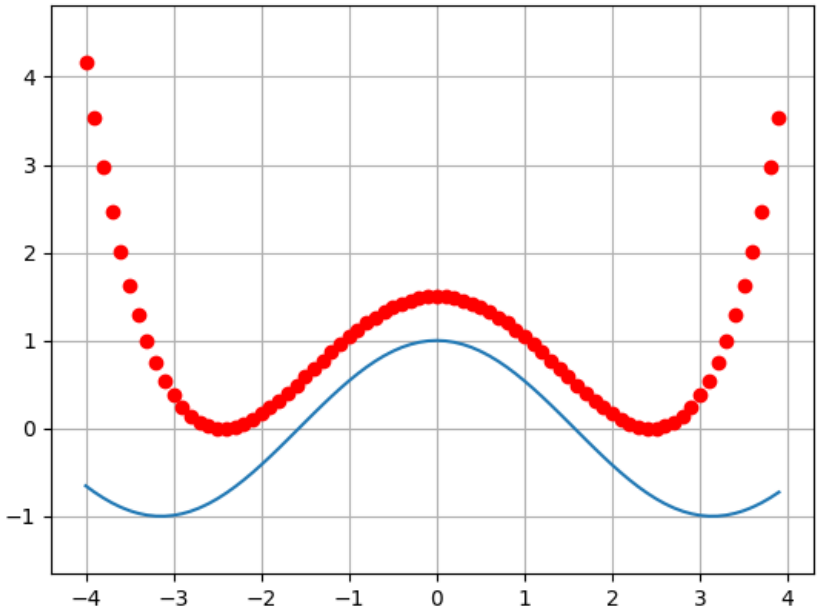

Figure 2: Cosine plotted in blue. The approximation (shifted slightly up) where $\tau = 2$ is in red.

to compare $\cos$ and the approximation. We can observe the approximation is excellent when inside the interval [-1.2,1.2]. We will utilize the numpy module (we'll study this in a week) to help us iterate over floats. The following code:

```
1
2  def cos_(x,tau):
3      def term(x,n):
4          pass
5      pass
6
7  x = np.arange(-4,4,.5)
8
9  for i in x:
10     print(math.cos(i),cos_(i,2),cos_(i,3),cos_(i,4))
```

x	$\cos(x)$	approximation
-4.0	-0.6536436208636119	3.666666666666666
-3.9	-0.7259323042001402	3.0343374999999986
-2.2999999999999985	-0.6662760212798231	-0.47899583333333284
-2.1999999999999984	-0.5885011172553444	-0.4439333333333325
-2.0999999999999983	-0.504846104599856	-0.39466249999999914
-1.9999999999999982	-0.4161468365471408 -	-0.33333333333333215
-1.8999999999999981	-0.32328956686350163	-0.2619958333333319
-1.799999999999998	-0.22720209469308517	-0.18259999999999849
-1.699999999999998	-0.12884449429552267	-0.09699583333333156
-1.299999999999997	0.26749882862458974	0.27400416666666894
-1.1999999999999975	0.3623577544766759	0.36640000000000233
-1.0999999999999974	0.4535961214255797	0.4560041666666689
-0.09999999999999654	0.9950041652780262	0.995004166666667

Table 1: A selection of values to show the accuracy in the interval for $\cos$ and its approximation.

produces:

```
1  -0.6536436208636119 3.666666666666666 -2.022222222222223 ↩
       -0.39682539682539764
2  -0.9364566872907963 1.127604166666667 -1.4255425347222221 ↩
       -0.8670416937934027
3  -0.9899924966004454 -0.125 -1.1375 -0.9747767857142857
4  -0.8011436155469337 -0.49739583333333326 -0.8364800347222221 ↩
       -0.7986358158172122
5  -0.4161468365471424 -0.33333333333333337 -0.4222222222222223 ↩
       -0.41587301587301595
6  0.0707372016677029 0.0859375 0.0701171875 0.07075282505580358
7  0.5403023058681398 0.5416666666666666 0.5402777777777777 ↩
       0.5403025793650793
8  0.8775825618903728 0.8776041666666666 0.8775824652777777 ↩
       0.8775825621589781
9  1.0 1.0 1.0 1.0
10 0.8775825618903728 0.8776041666666666 0.8775824652777777 ↩
       0.8775825621589781
11 0.5403023058681398 0.5416666666666666 0.5402777777777777 ↩
       0.5403025793650793
12 0.0707372016677029 0.0859375 0.0701171875 0.07075282505580358
13 -0.4161468365471424 -0.33333333333333337 -0.4222222222222223 ↩
       -0.41587301587301595
14 -0.8011436155469337 -0.49739583333333326 -0.8364800347222221 ↩
       -0.7986358158172122
15 -0.9899924966004454 -0.125 -1.1375 -0.9747767857142857
```

```
16  -0.9364566872907963 1.127604166666667 -1.4255425347222221 ↩
        -0.8670416937934027
```

Observe the last column is the best approximation.

Deliverables for Problem 6

- Complete the functions.
- The precision for this problem is critical, so make sure the returned values match those shown in the problem description.